

Informe de Laboratorio N.º 07

Asignatura: Mantenimiento, Configuración, Despliegue y Evolución de Software

Título de la práctica: El estándar IEEE 1219

Año lectivo: 2026-A

Semestre: IX

Integrante: Iván Rodrigo Huayhua Mayta

I. Solución de ejercicios/problemas

1. Sistema de software real seleccionado

Se selecciona **Apache HTTP Server (httpd)** como sistema de software real para el análisis del mantenimiento, debido a que es un sistema ampliamente utilizado, de larga vida útil y con evolución continua, por lo que resulta adecuado para ejemplificar la aplicación del estándar IEEE 1219 [1]. Además, su naturaleza de sistema maduro y extensible permite identificar problemas correctivos, adaptativos, perfectivos y preventivos, que son precisamente los tipos de mantenimiento que suelen abordarse en entornos reales [1].

2. Problemas o necesidades de mantenimiento identificados

De acuerdo con la definición de mantenimiento expuesta en el material del curso, el mantenimiento de software consiste en modificar un producto después de su entrega para corregir defectos, mejorar el rendimiento u otros atributos, o adaptarlo a cambios del entorno [1]. En el caso de Apache HTTP Server, se identifican las siguientes necesidades:

N.º	Tipo de mantenimiento	Necesidad identificada
1	Correctivo	Corrección de vulnerabilidades de seguridad y defectos funcionales detectados en producción.
2	Adaptativo	Adecuación a nuevos protocolos, bibliotecas criptográficas y cambios del entorno de despliegue.
3	Perfectivo	Mejora del rendimiento, de la escalabilidad y de la eficiencia en escenarios de alta concurrencia.
4	Preventivo	Refactorización de componentes con deuda técnica y fortalecimiento de pruebas para evitar fallos futuros.

Estas necesidades son coherentes con la realidad de un sistema legado o maduro, donde el mantenimiento no se limita a corregir errores, sino que también busca preservar la operatividad, la compatibilidad y la calidad evolutiva del software [1].

3. Relación de las fases del estándar IEEE 1219 con un posible plan de mantenimiento

El material analizado resume que el proceso de mantenimiento del estándar IEEE 1219 se organiza en las siguientes fases: identificación y clasificación del problema o modificación, análisis, diseño, implementación, pruebas del sistema, pruebas de aceptación y liberación del producto [1]. A partir de ello, puede formularse el siguiente plan de mantenimiento para Apache HTTP Server.

3.1 Identificación y clasificación del problema o modificación

En esta fase se registra formalmente el incidente, se le asigna un identificador, se clasifica el tipo de mantenimiento y se establece una prioridad inicial [1]. Para Apache, un caso representativo sería una vulnerabilidad de seguridad en un módulo como `mod_proxy`, clasificada como mantenimiento correctivo de alta prioridad, con impacto potencial en la confidencialidad o integridad del servicio.

Productos esperados de la fase:

- Relación de problemas o requerimientos.
- Clasificación del tipo de mantenimiento.
- Prioridad inicial.
- Estimación preliminar de recursos [1].

3.2 Análisis

Durante el análisis se determina la viabilidad técnica del cambio, su impacto funcional y los riesgos asociados [1]. En el sistema seleccionado, esto implicaría revisar dependencias con módulos relacionados, identificar efectos colaterales y definir una estrategia de pruebas orientada a evitar regresiones en funcionalidades críticas como autenticación, proxy y cifrado.

Productos esperados de la fase:

- Informe de viabilidad.
- Análisis detallado del cambio.
- Actualización de requisitos.
- Estrategia de pruebas.
- Plan de implementación [1].

3.3 Diseño

La fase de diseño parte de la documentación y del sistema existente para identificar los módulos a modificar y documentar la solución propuesta [1]. En Apache, esto podría involucrar la revisión del diseño interno del módulo afectado, las interfaces con otros componentes y la explicitación de restricciones técnicas, por ejemplo, evitar romper compatibilidad con configuraciones previas.

Productos esperados de la fase:

- Lista revisada de modificaciones.
- Diseño actualizado.
- Planes de prueba actualizados.
- Restricciones y riesgos documentados [1].

3.4 Implementación

La implementación comprende la codificación, las pruebas unitarias, la integración y la revisión de riesgos [1]. Dado que IEEE 1042-1987 describe la gestión de configuración como una disciplina que controla elementos, versiones, revisiones, autorizaciones de cambio, liberaciones y líneas base, esta fase debe ejecutarse bajo control de configuración formal [2]. Eso implica registrar el cambio, asociarlo a una solicitud aprobada, mantener trazabilidad del código y preservar la integridad de la línea base del producto [2].

3.5 Pruebas del sistema

Las pruebas del sistema incluyen pruebas funcionales, de interfaz y de regresión, con el fin de verificar que la modificación corrige el problema sin degradar otras partes del sistema [1]. Esta fase es especialmente importante cuando un mismo módulo comparte funciones con componentes críticos, ya que una corrección local podría introducir fallos en otras rutas de ejecución [1].

3.6 Pruebas de aceptación

Las pruebas de aceptación son realizadas por el cliente, por el usuario o por un tercero designado, para validar que la modificación satisface las necesidades planteadas y que tanto el software como su documentación resultan aceptables [1]. En un entorno real, esto equivaldría a desplegar la versión candidata en un entorno controlado y validar que el defecto fue corregido de manera satisfactoria.

3.7 Liberación del producto

La liberación del producto comprende actividades como auditoría de configuración física, notificación a usuarios, actualización de documentación, resguardo de versiones y puesta bajo control de la gestión de configuración del software [1]. IEEE 1042-1987 refuerza que la gestión de configuración proporciona visibilidad, control del cambio, control de bibliotecas, contabilidad de estado y procesamiento de liberaciones durante el ciclo de vida del software [2]. Por ello, la corrección no debe publicarse como un cambio aislado e informal, sino como una versión controlada, identificada y documentada [2].

4. Evidencias

Como evidencia conceptual del análisis realizado, se han empleado los siguientes elementos del material proporcionado:

- La definición de mantenimiento y la enumeración de fases del estándar IEEE 1219 [1].

- La relación entre el mantenimiento y la necesidad de pruebas de regresión, captura de problemas, seguimiento del sistema, correcciones y parches [1].
- La descripción de IEEE 1042-1987 sobre gestión de configuración, líneas base, solicitudes de cambio, autorizaciones de cambio y control de versiones y revisiones [2].

II. Solución del cuestionario

1. Caso de estudio: FINCONTROL

La empresa Finanzas Globales S.A. utiliza desde hace siete años el sistema FINCONTROL, desarrollado en .NET, para gestionar préstamos, pagos y reportes financieros; además, el laboratorio indica que el sistema es crítico, opera en más de 50 agencias en tiempo real y se conecta a la base central del banco [3]. El problema reportado consiste en la generación de reportes fiscales duplicados bajo ciertas condiciones, lo que afecta la información remitida a entes reguladores y exige una corrección urgente sin comprometer módulos críticos como el de pagos [3].

Las condiciones actuales descritas en el laboratorio incrementan el riesgo técnico del mantenimiento: no existen pruebas automatizadas, el código fue modificado sin control de versiones adecuado, el equipo actual no participó en el desarrollo original y el módulo de reportes comparte funciones con otros módulos críticos [3]. Desde la perspectiva de IEEE 1219, este escenario exige un proceso de mantenimiento disciplinado y fuertemente apoyado por prácticas de análisis de impacto, control de configuración y pruebas de regresión [1][2].

2. Selección de dos formularios para recabar información

De la lista propuesta en el laboratorio, los dos formularios más adecuados para un proyecto de mantenimiento sobre FINCONTROL son los siguientes [3]:

1. Solicitud de Cambio del Software/Sistema — IEEE 1042-1987.
2. Informe de incidencias de pruebas — IEEE 829-1983.

2.1 Solicitud de Cambio del Software/Sistema — IEEE 1042-1987

IEEE 1042-1987 incluye como parte de sus ejemplos y anexos el formulario **System/Software Change Request (SCR)**, el cual sirve para formalizar una petición de cambio sobre un producto controlado por gestión de configuración [2]. Este documento es el más adecuado para iniciar el mantenimiento de FINCONTROL, porque permite registrar el identificador del cambio, la descripción del problema, el impacto observado, la prioridad, los elementos afectados y la trazabilidad necesaria para su evaluación y aprobación [2].

En el caso planteado, el SCR resulta esencial porque el sistema es crítico, el defecto tiene implicancias regulatorias y existe riesgo de afectar módulos compartidos [3]. Además, la falta de control de versiones adecuado hace aún más importante disponer de una solicitud formal que documente el origen del cambio y sirva de base para análisis, aprobación e implementación bajo control [2][3].

2.2 Informe de incidencias de pruebas — IEEE 829-1983

El estándar IEEE 829 describe la documentación de pruebas y reconoce el **test incident report** como el documento utilizado para registrar eventos ocurridos durante la ejecución de pruebas que requieren investigación posterior [5]. Este formulario es especialmente pertinente en FINCONTROL, porque el defecto se presenta solo en determinadas condiciones y debe documentarse con precisión cuándo ocurre, bajo qué entradas se reproduce y cuáles fueron los resultados observados frente a los esperados [5].

Su utilidad es doble: por un lado, ayuda a reproducir técnicamente el defecto; por otro, aporta evidencia objetiva para las pruebas del sistema y de regresión luego de aplicar la corrección [1] [5]. En ausencia de pruebas automatizadas, la disciplina documental de incidencias cobra un valor todavía mayor para asegurar repetibilidad y trazabilidad del proceso de verificación.

3. Propuesta de diligenciamiento de los formularios

3.1 Solicitud de Cambio del Software/Sistema (SCR)

Campo	Contenido propuesto
Identificador	SCR-FINCONTROL-2026-001
Sistema	FINCONTROL
Módulo afectado	Reportes fiscales
Tipo de mantenimiento	Correctivo
Prioridad	Crítica
Descripción del problema	Se generan reportes fiscales duplicados en determinadas condiciones operativas, afectando la información remitida a entes reguladores.
Impacto	Riesgo regulatorio, inconsistencia de datos y posible afectación a módulos compartidos.
Entorno	Sistema .NET con operación en más de 50 agencias y conexión a base central.
Riesgos asociados	Posible regresión en el módulo de pagos; impacto sobre funciones compartidas; indisponibilidad parcial del servicio.
Solicitante	Área de TI / operación del sistema
Estado inicial	Pendiente de análisis y autorización

Este formulario permite alinear el inicio del cambio con las prácticas de control de configuración recomendadas por IEEE 1042-1987, incluyendo identificación, trazabilidad y posterior autorización del cambio [2].

3.2 Informe de incidencias de pruebas

Campo	Contenido propuesto
Identificador	TIR-FINCONTROL-2026-001
Sistema bajo prueba	FINCONTROL
Caso de prueba	Generación de reportes fiscales en condiciones concurrentes
Condición de ejecución	Operación simultánea desde múltiples agencias sobre el mismo periodo fiscal
Resultado esperado	Se genera un único reporte por operación válida
Resultado observado	Se generan reportes duplicados
Severidad	Alta / crítica
Fecha y hora	Según ejecución registrada
Evidencia	Logs, capturas de salida, registros de base de datos, identificadores de transacción
Observaciones	Posible condición de carrera o deficiencia de validación de duplicidad

Este informe facilita documentar el comportamiento defectuoso antes y después del cambio, y sirve como insumo para las pruebas de regresión y la aceptación de la corrección [1][5].

4. Sustento técnico de la estrategia de mantenimiento para FINCONTROL

Si se relaciona el caso FINCONTROL con las fases del estándar IEEE 1219, el plan de mantenimiento sería el siguiente [1][3]:

Fase IEEE 1219	Aplicación al caso FINCONTROL
Identificación y clasificación	Registrar el defecto, asignar prioridad crítica y clasificarlo como mantenimiento correctivo.
Análisis	Evaluar impacto en reportes, pagos y funciones compartidas; revisar riesgos por falta de pruebas automatizadas y control de versiones.
Diseño	Definir la solución técnica, el aislamiento del cambio y la estrategia de pruebas.
Implementación	Corregir el defecto bajo control de configuración y asociando el cambio al SCR aprobado.
Pruebas del sistema	Ejecutar pruebas funcionales, de integración y de regresión sobre reportes y pagos.
Pruebas de aceptación	Validar con usuarios responsables que el defecto desapareció y que no se afectó la operación.
Liberación del producto	Emitir una nueva versión controlada, documentada y trazable.

Esta secuencia es adecuada porque atiende no solo la urgencia del defecto, sino también los factores de riesgo descritos en el laboratorio, los cuales exigen disciplina técnica para evitar una corrección que introduzca problemas mayores en un sistema crítico [1][3].

5. Investigación sobre versionamiento semántico

El **Versionamiento Semántico (Semantic Versioning o SemVer)** es una especificación de versionado de software que define un esquema numérico con la forma **MAJOR.MINOR.PATCH**, donde cada segmento comunica el tipo de cambio introducido en una versión [4]. La especificación 2.0.0 establece que un incremento en **MAJOR** indica cambios incompatibles con versiones anteriores, un incremento en **MINOR** representa nuevas funcionalidades compatibles hacia atrás y un incremento en **PATCH** corresponde a correcciones de errores compatibles hacia atrás [4]. La documentación oficial de npm resume el mismo criterio y destaca que SemVer ayuda a gestionar dependencias y expectativas de compatibilidad entre productores y consumidores de software [6].

5.1 Interpretación de la estructura MAJOR.MINOR.PATCH

Componente	Significado	Ejemplo de cambio
MAJOR	Cambios incompatibles con versiones anteriores	Eliminación o rediseño de una API pública
MINOR	Nuevas funcionalidades compatibles hacia atrás	Incorporación de una característica adicional sin romper compatibilidad
PATCH	Corrección de errores compatible hacia atrás	Solución de un defecto sin alterar la interfaz pública

SemVer es especialmente útil en mantenimiento porque comunica, de manera compacta, el alcance del cambio entregado y el nivel de riesgo esperado al actualizar un sistema [4][6]. En proyectos con múltiples equipos o con dependencias externas, este mecanismo reduce ambigüedades y mejora la coordinación entre desarrollo, pruebas, operaciones y usuarios [6].

5.2 Ejemplos reales y conocidos

A continuación se presentan ejemplos reales y ampliamente conocidos de aplicación de versionamiento semántico:

Proyecto	Ejemplo	Interpretación
React	17.0.2 → 18.0.0	Cambio mayor asociado a una nueva versión principal con cambios relevantes de comportamiento y adopción [7].
Django	4.2.0 → 4.2.1	Corrección compatible hacia atrás dentro de la misma serie menor [8].
npm	10.2.3 → 10.2.4	Ajuste de mantenimiento o corrección de errores dentro de la misma línea funcional [6].
Software interno FINCONTROL	3.2.0 → 3.2.1	Corrección de bug sin romper compatibilidad funcional con clientes o módulos consumidores.

En el caso específico de FINCONTROL, la corrección del defecto de reportes duplicados debería versionarse razonablemente como un incremento **PATCH**, por ejemplo de 3.2.0 a 3.2.1, siempre que la solución no modifique contratos funcionales externos ni cambie interfaces consumidas por otros módulos. Si además se introdujeran nuevas capacidades compatibles,

correspondería un incremento **MINOR**; si se produjeran cambios incompatibles, entonces sería necesario un incremento **MAJOR** [4][6].

III. Conclusiones

1. El estándar IEEE 1219 organiza el mantenimiento de software en un proceso disciplinado de siete fases, lo que permite tratar cambios correctivos, adaptativos, perfectivos y preventivos con trazabilidad y control técnico [1].
2. La gestión de configuración descrita en IEEE 1042-1987 es un soporte indispensable para el mantenimiento, porque aporta mecanismos de identificación, control de cambios, líneas base, autorizaciones y liberaciones controladas [2].
3. En el caso FINCONTROL, la combinación de criticidad operativa, ausencia de pruebas automatizadas, deficiencias de versionado y funciones compartidas hace imprescindible una estrategia formal de mantenimiento basada en análisis de impacto, solicitudes de cambio y pruebas de regresión [1][2][3].
4. Los formularios más adecuados para recabar información en el caso planteado son la **Solicitud de Cambio del Software/Sistema** y el **Informe de incidencias de pruebas**, ya que cubren tanto la formalización del cambio como la evidencia necesaria para reproducir, verificar y cerrar el defecto [2][5].
5. El versionamiento semántico constituye una práctica complementaria altamente recomendable para FINCONTROL, dado que mejora la comunicación del alcance de cada entrega y fortalece la gobernanza de versiones en procesos de mantenimiento evolutivo y correctivo [4][6].

Referencias

- [1] G. Cabrera Málaga, *Mantenimiento, Configuración, Despliegue y Evolución de Software: El estándar IEEE 1219*. Arequipa, Perú: Universidad Nacional de San Agustín de Arequipa, 2026.
- [2] IEEE, *IEEE Guide to Software Configuration Management*, ANSI/IEEE Std 1042-1987. New York, NY, USA: Institute of Electrical and Electronics Engineers, 1988.
- [3] Universidad Nacional de San Agustín de Arequipa, *MCDES-Lab-07: Informe de Laboratorio - El estándar IEEE 1219*, documento de práctica, 2026.
- [4] T. Preston-Werner, "Semantic Versioning 2.0.0," *SemVer*, Jun. 2013. [En línea]. Disponible: <https://semver.org>. [Accedido: 19-jun-2026].
- [5] IEEE, *IEEE Standard for Software Test Documentation*, IEEE Std 829-1983. New York, NY, USA: Institute of Electrical and Electronics Engineers, 1983.
- [6] npm, Inc., "About semantic versioning," *npm Docs*. [En línea]. Disponible: <https://docs.npmjs.com/about-semantic-versioning/>. [Accedido: 19-jun-2026].
- [7] React Team, "React v18.0," *React Documentation*. [En línea]. Disponible: <https://react.dev>. [Accedido: 19-jun-2026].

[8] Django Software Foundation, “Django documentation,” *Django Project*. [En línea]. Disponible: <https://docs.djangoproject.com/>. [Accedido: 19-jun-2026].